# Database Systems(CSL3050)

## Lab Assignment 5 - Report

Soumen Kumar
Roll No-B22ES006

## 1 Introduction

This report contains descriptions and results of various PL/SQL queries executed on a database. Each query addresses different aspects of PL-SQL, such as data retrieval, aggregation, and ranking. The screenshots of the query outputs are provided for clarity.

## Query 1: Inserting Triangular Numbers into a Column

**Description:**
This query uses a `FOR` loop to insert the first 10 numbers of the Triangular Number sequence into a column of an output table.

**Code:**

```
CREATE TABLE test_table (
    id NUMBER
);
BEGIN
    FOR i IN 1..10 LOOP
        DECLARE
            triangular_number NUMBER;
        BEGIN
            triangular_number := i * (i + 1) / 2;
            INSERT INTO test_table (id) VALUES (triangular_number);
            DBMS_OUTPUT.PUT_LINE('Inserted triangular number: ' || triangular_number);
        END;
    END LOOP;
END;
/
select * from test_table;
```

**Diagram:**

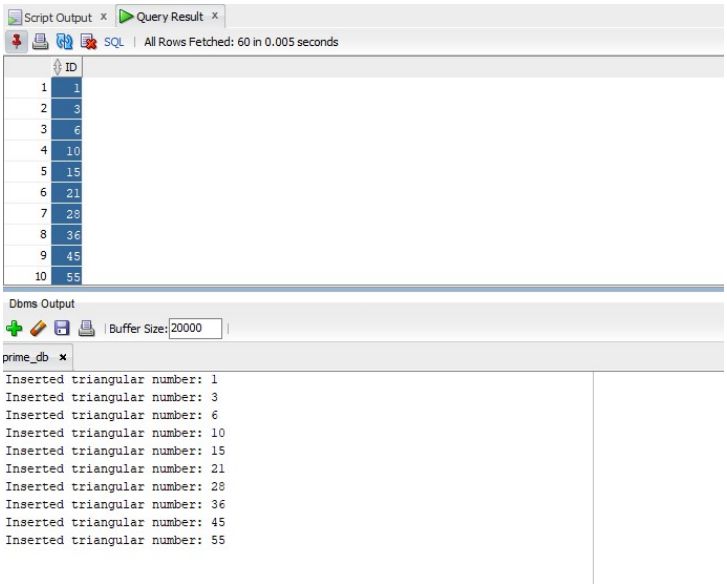



Figure 1: Triangular Numbers Sequence Visualization

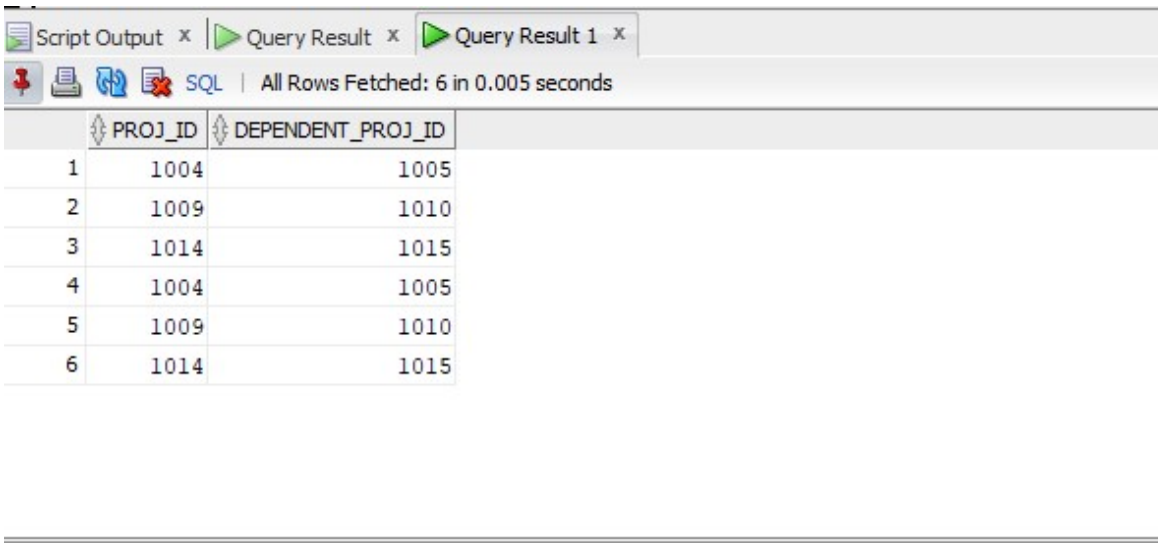
# Query 2: Insert Pairs From Dependency Table

**Description:**
This query uses a `CURSOR`, `FOR` loop, and `IF` statement to insert specific pairs (proj_id, dependent_proj_id) from the `dependency` table where dependent_proj_id is in [3005, 3010, 3050].

**Code:**

```
CREATE TABLE specific_dependencies (
    proj_id NUMBER,
    dependent_proj_id NUMBER
);
DECLARE
    CURSOR dep_cursor IS
        SELECT proj_id, dependent_proj_id
        FROM dependency
        WHERE dependent_proj_id IN (1005, 1010, 1015);
BEGIN
    FOR rec IN dep_cursor LOOP
        INSERT INTO specific_dependencies (proj_id, dependent_proj_id)
        VALUES (rec.proj_id, rec.dependent_proj_id);
    END LOOP;
    COMMIT;
END;
/
select * from specific_dependencies;
```

**Diagram:**




| | PROJ_ID | DEPENDENT_PROJ_ID |
|---|---|---|
| 1 | 1004 | 1005 |
| 2 | 1009 | 1010 |
| 3 | 1014 | 1015 |
| 4 | 1004 | 1005 |
| 5 | 1009 | 1010 |
| 6 | 1014 | 1015 |

Figure 2: Dependency Pairs Insertion

# Query 3: Counting Employees by Division

**Description:**

This query uses a `WHILE` loop to display the number of employees working in each division.

**Code:**

```
DECLARE
    v_div_name division.division_name%TYPE;
    v_count NUMBER;
    CURSOR emp_cursor IS
        SELECT division_name
        FROM division;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO v_div_name;
        EXIT WHEN emp_cursor%NOTFOUND;

        SELECT COUNT(*) INTO v_count
        FROM employee
        WHERE division_name = v_div_name;

        DBMS_OUTPUT.PUT_LINE(v_div_name || ': ' || v_count);
    END LOOP;
    CLOSE emp_cursor;
END;
/
```

**Diagram:**

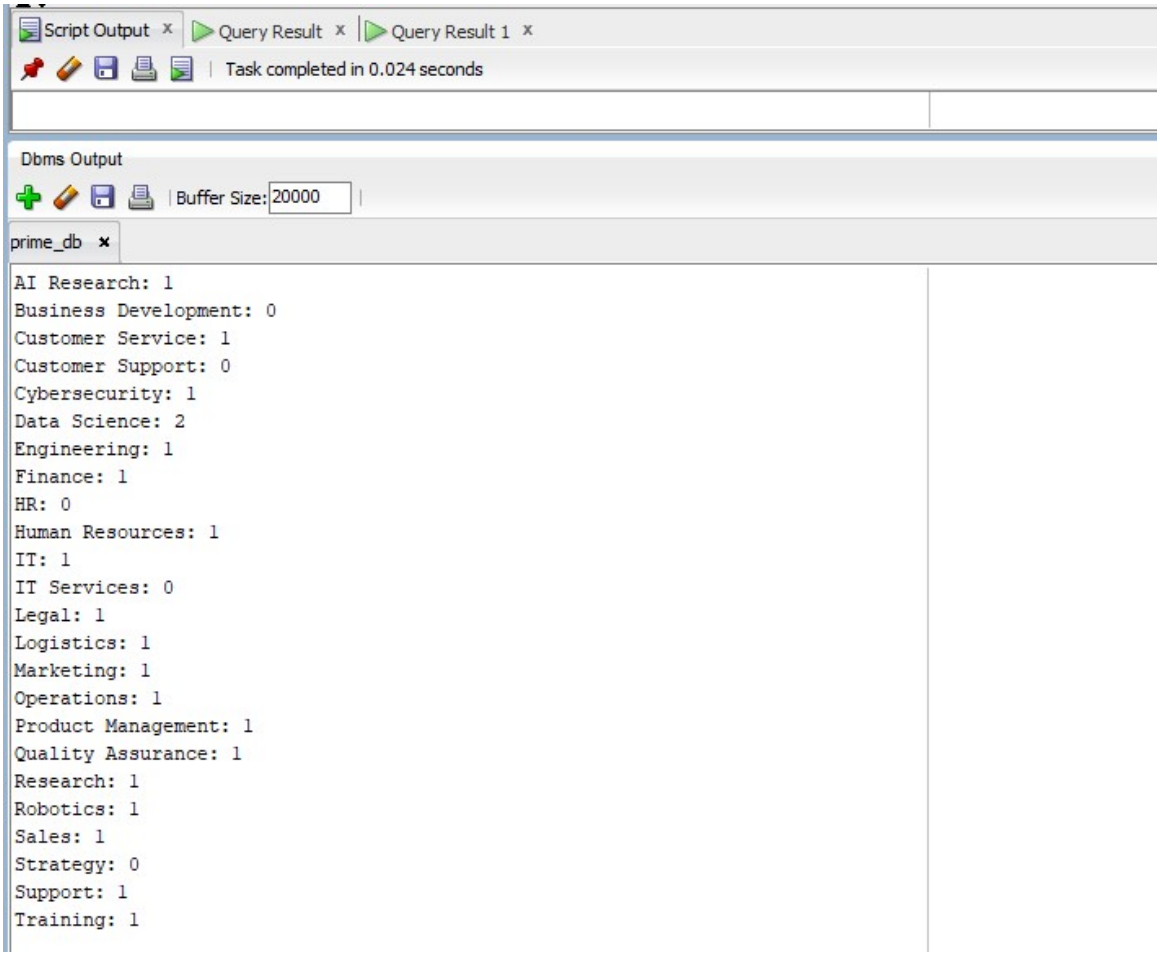



Figure 3: Employee Count Per Division

# Query 4: Update Completion Status for Trainees in AI Research

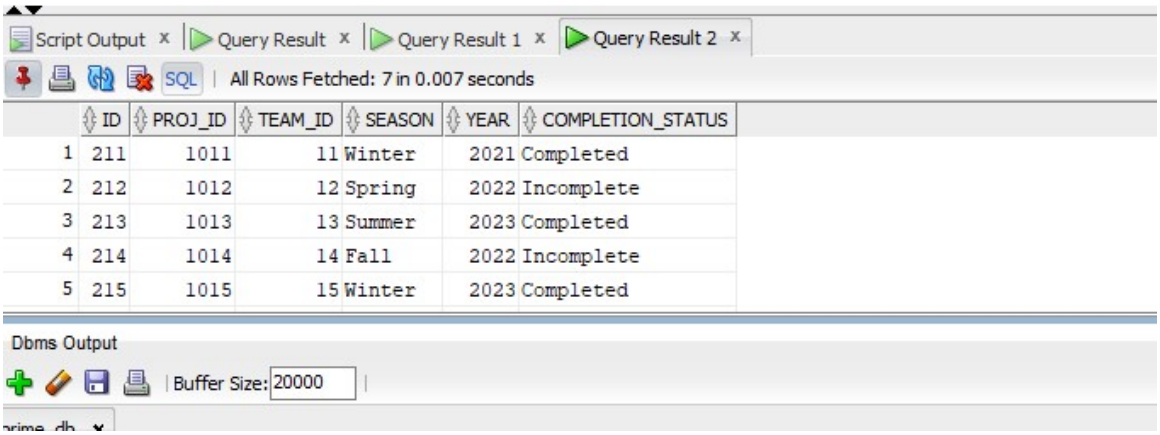
**Description:**
This query uses an `UPDATE` statement to change the completion status of trainees marked as "Incomplete" for the project 'AI Research'.

**Code:**

```
BEGIN
    UPDATE enrollment
    SET completion_status = 'Completed'
    WHERE proj_id IN (
        SELECT proj_id
        FROM project
        WHERE proj_title = 'AI Research'
    )
    AND completion_status = 'Incomplete';

    COMMIT;
END;
/
```

**Diagram:**




| | ID | PROJ_ID | TEAM_ID | SEASON | YEAR | COMPLETION_STATUS |
|---|---|---|---|---|---|---|
| 1 | 211 | 1011 | 11 | Winter | 2021 | Completed |
| 2 | 212 | 1012 | 12 | Spring | 2022 | Incomplete |
| 3 | 213 | 1013 | 13 | Summer | 2023 | Completed |
| 4 | 214 | 1014 | 14 | Fall | 2022 | Incomplete |
| 5 | 215 | 1015 | 15 | Winter | 2023 | Completed |

Figure 4: Completion Status Update for AI Research Project

# Query 5: Increase Operating Budget by 12%

**Description:**

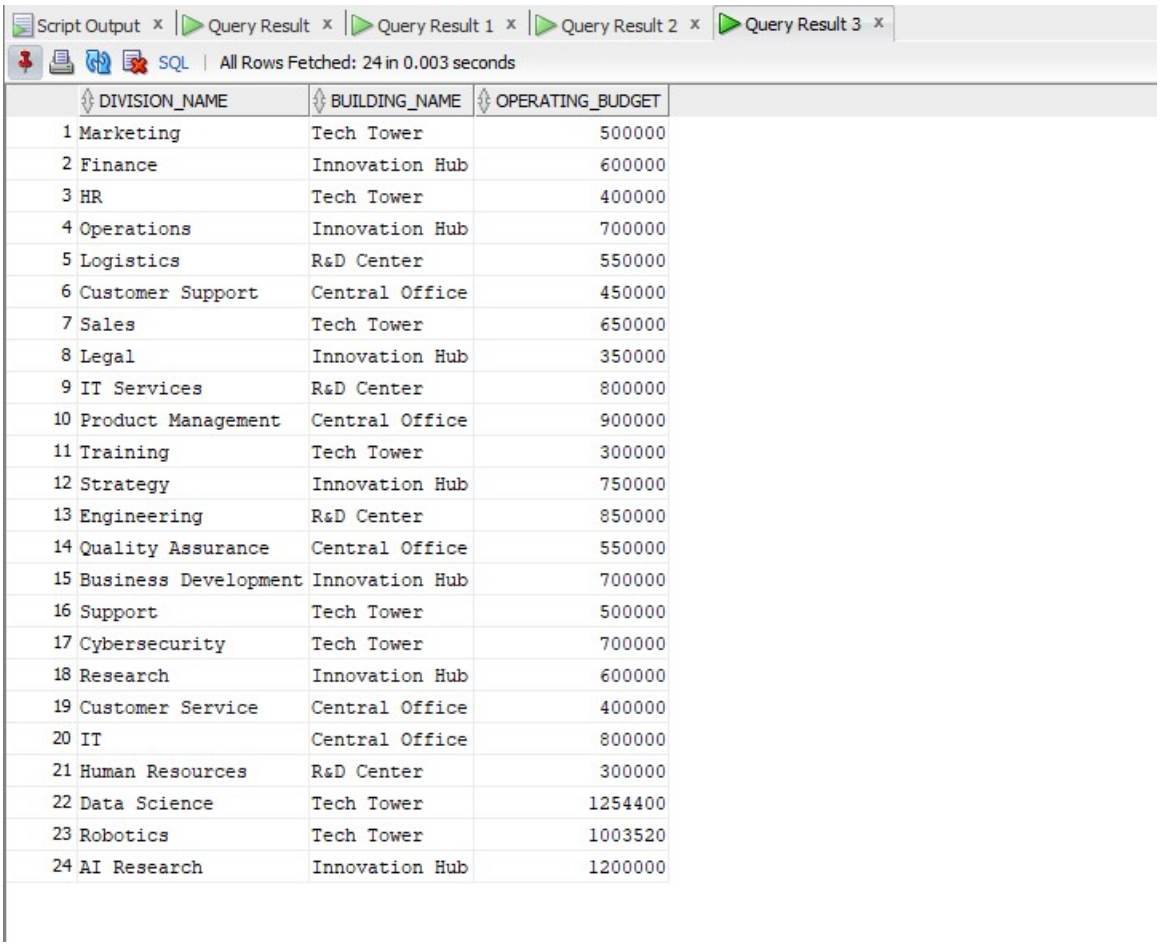
This query uses an `UPDATE` statement to increase the operating budget of the Data Science and Robotics divisions by 12%.

**Code:**

```
BEGIN
    UPDATE division
    SET operating_budget = operating_budget * 1.12
    WHERE division_name IN ('Data Science', 'Robotics');

    COMMIT;
END;
/
```

**Diagram:**



Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3

SQL | All Rows Fetched: 24 in 0.003 seconds

| | DIVISION_NAME | BUILDING_NAME | OPERATING_BUDGET |
|---|---|---|---|
| 1 | Marketing | Tech Tower | 500000 |
| 2 | Finance | Innovation Hub | 600000 |
| 3 | HR | Tech Tower | 400000 |
| 4 | Operations | Innovation Hub | 700000 |
| 5 | Logistics | R&D Center | 550000 |
| 6 | Customer Support | Central Office | 450000 |
| 7 | Sales | Tech Tower | 650000 |
| 8 | Legal | Innovation Hub | 350000 |
| 9 | IT Services | R&D Center | 800000 |
| 10 | Product Management | Central Office | 900000 |
| 11 | Training | Tech Tower | 300000 |
| 12 | Strategy | Innovation Hub | 750000 |
| 13 | Engineering | R&D Center | 850000 |
| 14 | Quality Assurance | Central Office | 550000 |
| 15 | Business Development | Innovation Hub | 700000 |
| 16 | Support | Tech Tower | 500000 |
| 17 | Cybersecurity | Tech Tower | 700000 |
| 18 | Research | Innovation Hub | 600000 |
| 19 | Customer Service | Central Office | 400000 |
| 20 | IT | Central Office | 800000 |
| 21 | Human Resources | R&D Center | 300000 |
| 22 | Data Science | Tech Tower | 1254400 |
| 23 | Robotics | Tech Tower | 1003520 |
| 24 | AI Research | Innovation Hub | 1200000 |

Figure 5: Budget Increase for Data Science and Robotics

# Query 6: Projects on Different Floors

**Description:**
This query creates an output table listing divisions where projects are conducted by teams on at least 2 different floors in the same building.

**Code:**

```
CREATE TABLE multi_floor_projects (
    division_name VARCHAR2(50)
);

BEGIN
    INSERT INTO multi_floor_projects (division_name)
    SELECT DISTINCT d.division_name
    FROM division d
    JOIN project p ON d.division_name = p.division_name
    JOIN assignment a ON p.proj_id = a.proj_id
    JOIN workspace w ON a.building_name = w.building_name
    GROUP BY d.division_name
    HAVING COUNT(DISTINCT a.floor_number) > 1;

    COMMIT;
END;
/
select * from multi_floor_projects;
```

**Diagram:**

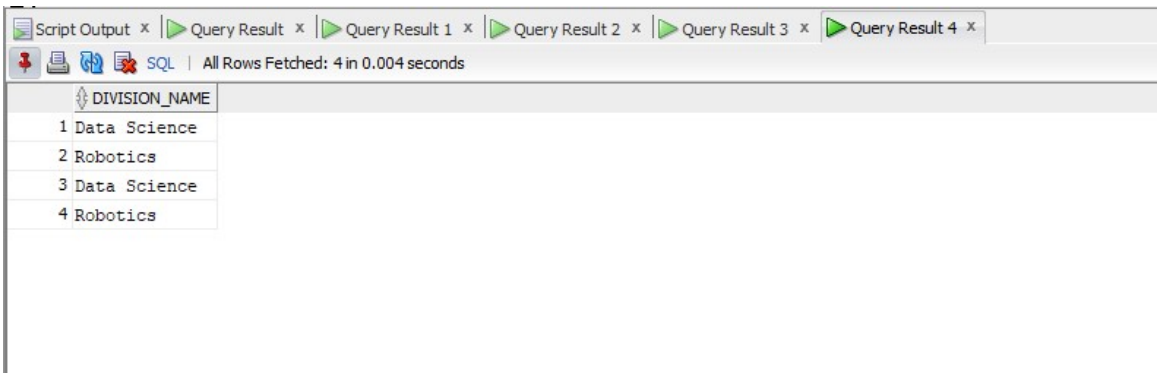



Figure 6: Projects Conducted on Multiple Floors

# Query 7: Jane Doe's Project Performance

**Description:**
This query uses a `RECORD` to display the completion status and duration of each project Jane Doe worked on in the last 4 seasons.

**Code:**

```
DECLARE
    TYPE performance_rec IS RECORD (
        completion_status VARCHAR2(50),
        duration_months NUMBER
    );

    v_rec performance_rec;
BEGIN
    FOR rec IN (
        SELECT e.completion_status, p.duration_months
        FROM works_on w
        JOIN enrollment e ON w.proj_id = e.proj_id
        JOIN project p ON w.proj_id = p.proj_id
        WHERE w.ID = 211 -- Jane Doe's ID
        AND e.season IN ('Spring', 'Summer', 'Fall', 'Winter')
        AND e.year >= EXTRACT(YEAR FROM SYSDATE) - 4
    ) LOOP
        v_rec.completion_status := rec.completion_status;
        v_rec.duration_months := rec.duration_months;

        DBMS_OUTPUT.PUT_LINE('(' || v_rec.completion_status || ', ' || v_rec.duration_months || ' months)'
    );
    END LOOP;
END;
/

END;
```

**Diagram:**

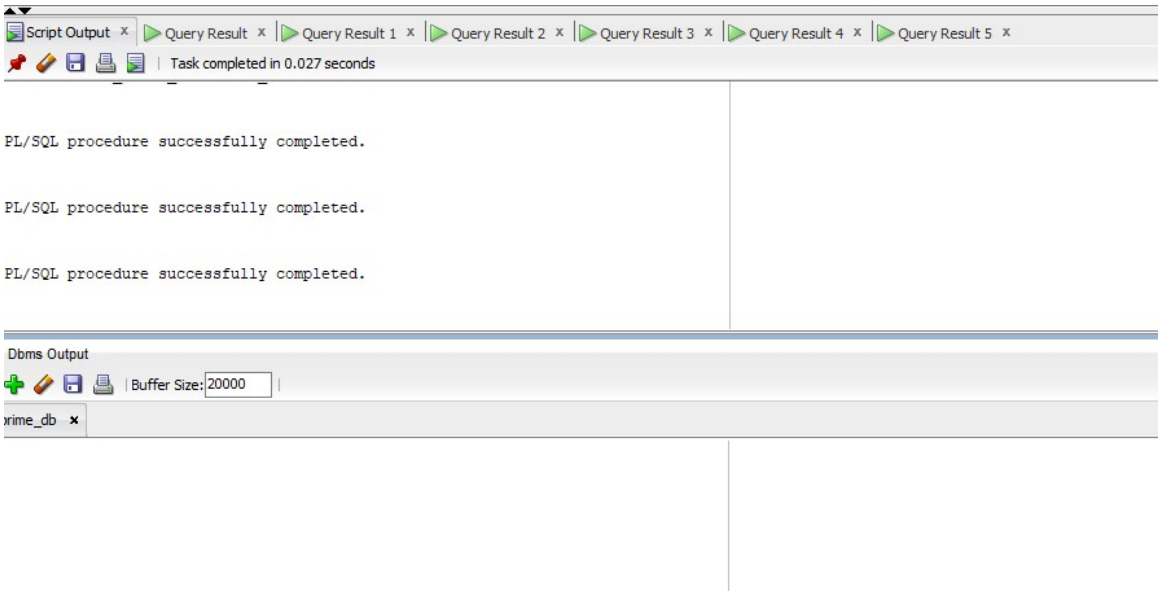



Figure 7: Jane Doe's Project Performance

# Query 8: Handling Missing Employee Exception

**Description:**

This query uses Exception Handling to manage the case where an employee named 'Amit Verma' is not found in the database.

**Code:**

```
DECLARE
    v_emp_name employee.emp_name%TYPE := 'Amit Verma';
    v_emp_id employee.ID%TYPE;
BEGIN
    SELECT ID INTO v_emp_id
    FROM employee
    WHERE emp_name = v_emp_name;

    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_emp_id);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee named ' || v_emp_name || ' not found.');
END;
/
```

**Diagram:**

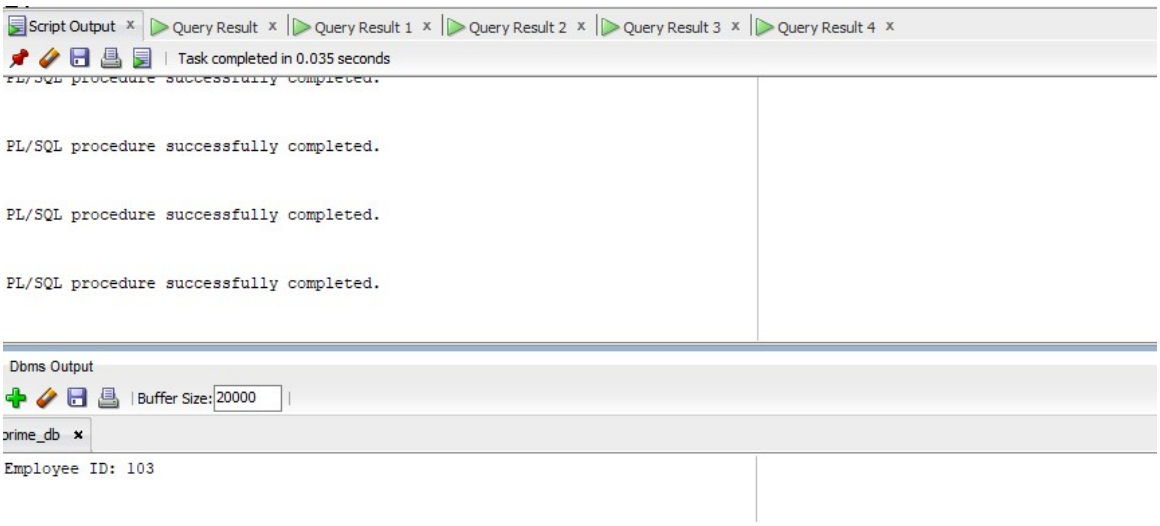



Figure 8: Exception Handling for Missing Employee

## Query 9: Projects with High Trainee Success Rates

**Description:**
This query lists the projects worked on by 'Prof. Suchetana Chakraborty' in the last 5 years where more than 90% of trainees achieved a successful completion status.

**Code:**

```
DECLARE
    CURSOR project_cursor IS
        SELECT p.proj_id, p.proj_title,
               COUNT(t.ID) AS total_trainees,
               SUM(CASE WHEN e.completion_status = 'Successful' THEN 1 ELSE 0 END) AS successful_trainees
        FROM project p
        JOIN enrollment e ON p.proj_id = e.proj_id
        JOIN mentor m ON e.ID = m.trainee_ID
        JOIN employee emp ON m.mentor_ID = emp.ID
        JOIN trainee t ON e.ID = t.ID
        WHERE emp.emp_name = 'Suchetana Chakraborty'
          AND p.duration_months <= 60  -- Projects done in the last 5 years (assumed in months)
        GROUP BY p.proj_id, p.proj_title;

    -- Variables to hold data from cursor
    v_proj_id project.proj_id%TYPE;
    v_proj_title project.proj_title%TYPE;
    v_total_trainees NUMBER;
    v_successful_trainees NUMBER;
    v_success_rate NUMBER;
BEGIN
        v_proj_id := project_rec.proj_id;
        v_proj_title := project_rec.proj_title;
        v_total_trainees := project_rec.total_trainees;
        v_successful_trainees := project_rec.successful_trainees;

        -- Calculate the success rate
        v_success_rate := (v_successful_trainees / v_total_trainees) * 100;

        -- If the success rate is greater than 90%, insert it into the output table
        IF v_success_rate > 90 THEN
            INSERT INTO output_table_suchetana_projects (proj_id, proj_title, success_rate)
            VALUES (v_proj_id, v_proj_title, v_success_rate);
        END IF;
    END LOOP;

    COMMIT;
END;

select * from output_table_suchetana_projects;
```

**Diagram:**

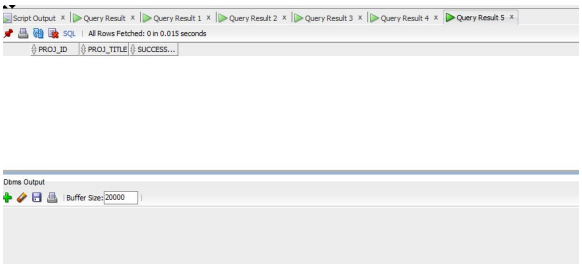



Figure 9: Projects with High Trainee Success Rates

## 2 Conclusion

This report demonstrates a variety of PL-SQL queries used to analyze and manage database information. Each query showcases different PL-SQL features and their practical applications in database management.